Nombre y Apellido: Legajo:

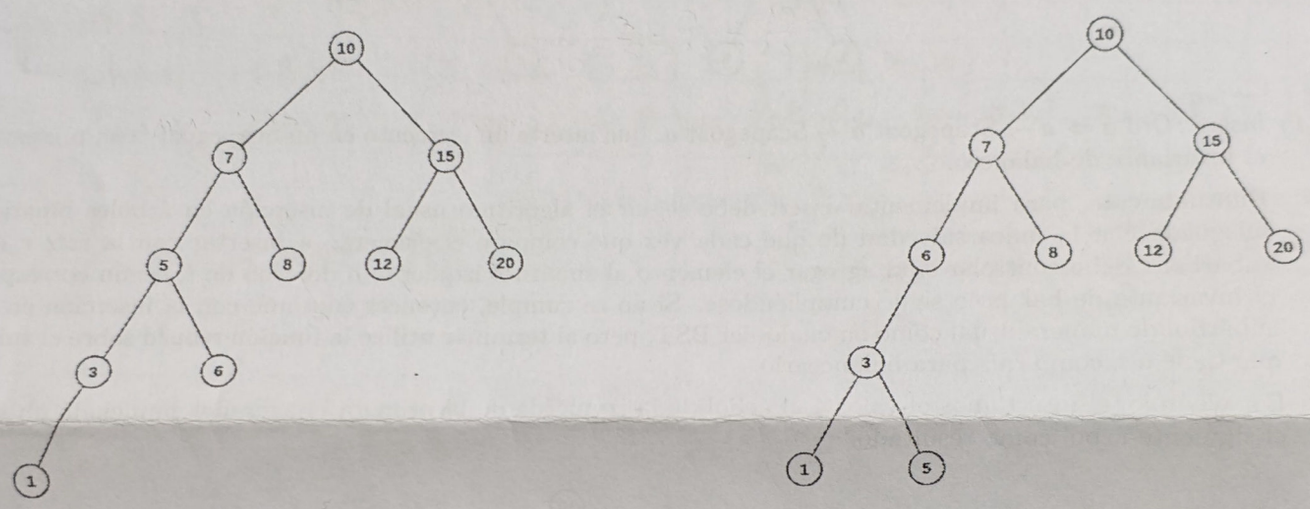
Examen Parcial 1

1. Un **Scapegoat tree** es un árbol binario de búsqueda t que verifica la siguiente propiedad: para todo nodo n del árbol se cumple que

$$\frac{size(l_n)}{size(t_n)} \leqslant \frac{2}{3} \text{ y } \frac{size(r_n)}{size(t_n)} \leqslant \frac{2}{3}$$

donde l_n y r_n son los hijos izquierdos y derechos, respectivamente, de n, t_n es el subárbol de t que tiene a n como raíz y la función $size$ determina el número de nodos de un árbol.

Por ejemplo, el árbol de la izquierda es un Scapegoat tree mientras que el de la derecha no lo es porque el nodo que tiene valor 6 no verifica la propiedad enunciada dado que $\frac{3}{4} \nleqslant \frac{2}{3}$:



Observar que esta propiedad sirve para balancear el árbol, de manera que la altura sea logarítmica en el número de nodos, lo que permite hacer eficiente la búsqueda de elementos. Como contrapartida, al insertar y eliminar elementos en el árbol, debemos garantizar que se preserve el invariante de balanceo.

Para representar este tipo de árboles en Haskell usaremos el siguiente tipo de datos:

```
data Scapegoat a = E | N a Int (Scapegoat a) (Scapegoat a) deriving (Eq, Show)
```

donde el entero del constructor N representa el número de nodos del árbol.

Definir eficientemente las siguientes funciones en Haskell:

a) size :: Scapegoat a → Int, que calcule el tamaño de un árbol (número de nodos).

b) isBST :: Ord a ⇒ Scapegoat a → Bool, que dado un valor de tipo Scapegoat a determine si es un árbol binario de búsqueda.

c) isScapegoatTree :: Ord a ⇒ Scapegoat a → Bool, que dado un valor de tipo Scapegoat a determine si es un Scapegoat tree.

d) member :: Ord a ⇒ a → Scapegoat a → Bool, que dado un Scapegoat tree y un elemento determine si el elemento está en el árbol.

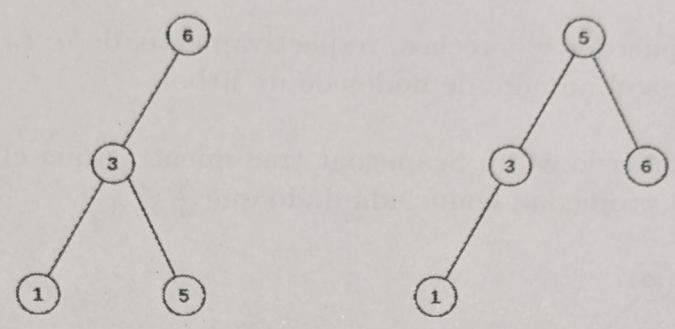
e) rebuild :: Scapegoat a → Scapegoat a, que dado un valor de tipo Scapegoat a, árbol binario de búsqueda, lo reestructure a otro árbol binario de búsqueda, con exactamente los mismos elementos pero casi perfectamente balanceado. Decimos que un árbol es casi perfectamente balanceado cuando para todo nodo del árbol la diferencia entre el número de nodos de su hijo izquierdo y derecho es a lo sumo 1.

Puntualmente, para implementar rebuild se pide:

i. Aplanar el árbol en una lista según el recorrido in order.

ii. Elegir al elemento del medio de la lista, *mid*, como raíz y usar como hijos izquierdo y derecho a los resultados de aplicar recursivamente este procedimiento sobre las sublistas que quedan determinadas por los elementos a izquierda y derecha de *mid*, respectivamente.

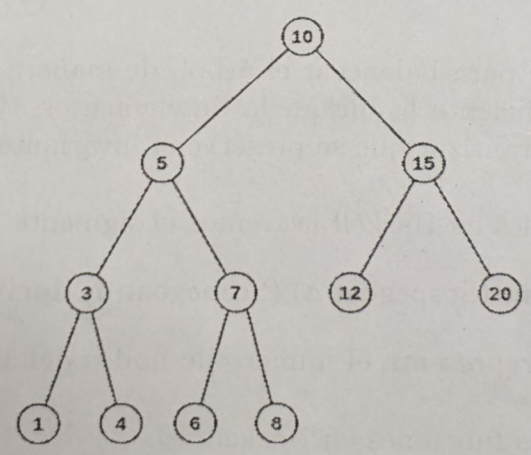
Por ejemplo, si aplicamos este procedimiento al árbol de la izquierda, en el primer paso obtenemos la lista $\langle 1, 3, 5, 6 \rangle$ y en el segundo paso obtenemos, como resultado final, el árbol de la derecha.



f) insert :: Ord $a \Rightarrow a \rightarrow$ Scapegoat $a \rightarrow$ Scapegoat a, que inserte un elemento en un Scapegoat tree, preservando el invariante de balanceo.

Puntualmente, para implementar insert debe seguir el algoritmo usual de inserción en árboles binarios de búsqueda con la única salvedad de que cada vez que compare el elemento a insertar con la raíz r de un subárbol t debe controlar si al agregar el elemento al subárbol izquierdo o derecho de t, según corresponda, el invariante de balanceo sigue cumpliéndose. Si no se cumple, entonces continúe con la inserción en dicho subárbol de manera usual como en cualquier BST, pero al terminar utilice la función rebuild sobre el subárbol que tiene a r como raíz para balancearlo.

Por ejemplo, si insertamos el valor 4 al árbol de la izquierda de la primera imagen del enunciado obtenemos el siguiente árbol como resultado:



Nótese que, en este ejemplo, fue necesario hacer un rebuild en el subárbol que tiene como raíz al valor 7 en el árbol original.

2. Dada la siguiente recurrencia:

$$T(n) = \begin{cases} 1 & n = 1 \\ 4T(\lfloor n/2 \rfloor) + n & n > 1 \end{cases}$$

a) Dar una cota asintótica Θ para T. No demuestre que la cota es correcta pero escriba brevemente el procedimiento que usó para hallarla.

b) Demostrar usando el método de substitución que dicha cota es una cota O de T.